

Big-O, Big-Omega, Big-Theta

Asymptotic notation compares growth rates of functions for large input sizes n .

Big-O: Asymptotic Upper Bound

Natural explanation: Big-O gives a ceiling. It says that after n gets large enough, $f(n)$ will not grow faster than some constant multiple of $g(n)$. In plain terms, $g(n)$ is an upper growth-rate model for $f(n)$.

Academic definition: We say $f(n) \in O(g(n))$ if there exist constants $c > 0$ and n_0 such that

$$0 \leq f(n) \leq c g(n) \quad \text{for all } n \geq n_0.$$

Interpretation: for sufficiently large n , f grows no faster than a constant multiple of g .

Typical use in algorithms: upper bound on running time (often worst-case).

![[big-o-analysis.webp]]

Big-Omega: Asymptotic Lower Bound

Natural explanation: Big-Omega gives a floor. It says that after n gets large enough, $f(n)$ cannot grow slower than a constant multiple of $g(n)$. So $g(n)$ is a guaranteed minimum growth rate for $f(n)$.

Academic definition: We say $f(n) \in \Omega(g(n))$ if there exist constants $c > 0$ and n_0 such that

$$0 \leq c g(n) \leq f(n) \quad \text{for all } n \geq n_0.$$

Interpretation: for sufficiently large n , f grows at least as fast as a constant multiple of g .

Typical use in algorithms: lower bound on running time (sometimes best-case, sometimes problem lower bounds depending on context).

![[big-omega-analysis.webp]]

Big-Theta: Asymptotically Tight Bound

Natural explanation: Big-Theta gives a tight band. It says $f(n)$ grows at the same rate as $g(n)$ up to constant factors, meaning $g(n)$ is neither too small nor too large as a model.

Academic definition: We say $f(n) \in \Theta(g(n))$ if there exist constants $c_1, c_2 > 0$ and n_0 such that

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \text{for all } n \geq n_0.$$

Equivalent view: f is both $O(g)$ and $\Omega(g)$.

Interpretation: f and g have the same asymptotic growth rate up to constant factors.

![[big-theta-analysis.png]]

Fast Intuition

- Ignore constant factors: $7n$ and n are same order.
- Ignore lower-order terms: $n^2 + 3n + 10$ behaves like n^2 for large n .
- Focus on dominant growth term as $n \rightarrow \infty$.

Examples

- $3n + 2 \in \Theta(n)$
- $5n^2 + n + 1 \in \Theta(n^2)$
- $n \log n \in O(n^2)$ and also $n \log n \in \Omega(n)$
- Dynamic array doubling (from Week 4):
 - A single resize can cost $O(n)$.
 - Over n pushes, total cost is $O(n)$.
 - Therefore amortized cost per push is $O(1)$.

How To Show a Bound (Practical Template)

To show $f(n) \in O(g(n))$: 1. Start from $f(n)$. 2. Find a simple inequality $f(n) \leq cg(n)$ for all $n \geq n_0$. 3. State explicit constants c, n_0 .

To show $f(n) \in \Omega(g(n))$: 1. Start from $f(n)$. 2. Find $cg(n) \leq f(n)$ for all $n \geq n_0$. 3. State explicit constants c, n_0 .

To show $f(n) \in \Theta(g(n))$: 1. Prove both $O(g(n))$ and $\Omega(g(n))$.

Common Pitfalls

- Writing “=” between a function and a class, e.g. $f(n) = O(n)$.
 - Better: $f(n) \in O(n)$.
 - Assuming Big-O alone gives a tight bound.
 - Mixing worst-case, average-case, and amortized analysis without saying which one is used.
-

Related

- [[Amortized Analysis]]
- [[Lectures/Week 4]]